A9 Testing

pytest, fixtures, local SparkSession and DataFrame equality checks.

A9-L1 pytest basics, and a SparkSession fixture shared by every test

One session-scoped fixture, and tests with known answers that fail for the bugs you care about.

```python
def test_weighted_share_uses_weights() -> None:
    assert weighted_share([1, 0], [3.0, 1.0]) == 0.75
```

```python
@pytest.fixture(scope="session")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    yield session
    session.stop()
```

```python
@pytest.mark.parametrize(("flags", "weights", "expected"), [([1], [2.0], 1.0), ([0, 0], [1.0, 1.0], 0.0)])
def test_share(flags, weights, expected): ...
```

What does pytest run?	Functions named test_* in files named test_*.py. Plain assert statements.
SparkSession fixture scope	scope="session": start Spark once per test run.
yield in a fixture	Hands the value to tests. Code after yield is the teardown.
What makes a test worth having?	It fails for the bug you fear: known answers that separate right from wrong, plus edge cases.

A9-L2 A fast local SparkSession and readable test data

A tuned local session makes tests fast enough to run on every change. Builders with explicit schemas make them short and exact.

```python
builder = (
    SparkSession.builder.master("local[2]")
    .config("spark.sql.shuffle.partitions", "2")
    .config("spark.ui.enabled", "false")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.sql.sources.default", "delta")
)
spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

```python
def respondent(**overrides: object) -> dict[str, object]:
    row = {"respondent_id": "R1", "wave_id": "W1", "final_weight": 1.0, "in_scope": True}
    row.update(overrides)
    return row
```

Local test session settings	master("local[2]"), shuffle partitions 2, UI off, Delta configured with configure_spark_with_delta_pip.
Default spark.sql.shuffle.partitions	200: far too many for test data.
Why explicit schemas in test data?	Inference fails on all-None columns and can pick different types from production.
Test data builder	A function with valid defaults. Tests override only the fields they are about, and typos raise.

A9-L3 Compare DataFrames properly, and test that MERGE is idempotent

One helper compares DataFrames the right way, and one small test proves a MERGE survives a retry and a repeated key.

```
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual

assertDataFrameEqual(actual, expected)
```

Compare two DataFrames in a test	pyspark.testing.assertDataFrameEqual(actual, expected). Ignores order, tolerates float rounding, checks schema.
When should row order matter?	Only for ranked output. Pass checkRowOrder=True.
Idempotence test for a MERGE	Load, snapshot with collect(), load again, assertDataFrameEqual(table, snapshot).
Why collect() for the snapshot?	A DataFrame is lazy and would re-read the table after the second run.